

Milestone 1 briefing — sequential SD-code baseline

This is the background for someone presenting slides 5–7 of `gPPM_Milestone2_Clean.pptx`. The slide-by-slide words to say are embedded as notes inside the PowerPoint (LibreOffice Impress: View → Notes). Read this briefing to understand what the milestone required, what was implemented, and what the numbers mean.

The assignment in one sentence

Build a correct, reproducible **sequential reference decoder** for SD erasure codes using the traditional matrix calculation order, then record its operation count and recovery throughput. Milestone 2 will compare against this exact reference.

The source paper is Li et al., *gPPM: A Generalized Matrix Operation and Parallel Algorithm to Accelerate the Encoding/Decoding Process of Erasure Codes*, ACM TACO 20(4), 2023. Milestone 1 implements the traditional method introduced in its background section; it is not yet the paper's PPM optimization.

What the decoder is recovering

A **sector** is a block of bytes stored on one disk. A **stripe** is the whole group encoded together: n disks and r sectors from each disk. In the paper's example $n=r=4$, so imagine a 4×4 grid of sixteen sectors. Sector $c = \text{row} * n + \text{disk}$, so sector 6 is row 1 on disk 2.

An SD code, written $\text{SD}^{\{m,s\}}_{\{n,r\}}(w \mid a_0, \dots, a_{m+s-1})$, is configured to handle m whole-disk failures plus s additional sector failures. w says whether each finite-field symbol has 8, 16, or 32 bits. The a values are published coding coefficients. The project loads those values from `data/FAST-Coefficients.txt`, Plank's SD coefficient table. It lists 3,969 parameter configurations, with $n, r = 4 \dots 24$ and $m, s = 1 \dots 3$. This table contains code settings, **not** real disk contents.

The code builds a parity-check matrix H with $m*r+s$ rows and $n*r$ columns. A column refers to one physical sector. The first $m*r$ rows are row-local equations: each touches one horizontal stripe row. The remaining s equations are dense and can involve the entire stripe. The encoder calculates parity sectors so the complete valid stripe B satisfies $H*B=0$.

What we had to implement

1. **GF arithmetic:** XOR for addition, finite-field multiplication for weighted parity; support all published field widths, $\text{GF}(2^8)$, $\text{GF}(2^{16})$, and $\text{GF}(2^{32})$. The expensive primitive, `mult_XORs`, multiplies a full source sector region by a coefficient and XORs into a destination region.
2. **SD matrix construction:** read the published coefficients and build H in the paper's layout (`src/sd_code.c`).
3. **Matrix utilities:** allocate/split matrices, invert a square failed matrix over the field, and count nonzero coefficients (`src/matrix.c`).
4. **Traditional recovery:** keep surviving columns as S , failed columns as F , compute $T=S*BS$, and then recover $BF=F^{-1}*T$ (`src/codec.c`, `ec_recover()`).
5. **Reference measurements:** count sector-sized operations and time sequential encoding and decoding using reproducible data (`src/sweep.c`, `src/benchmark.c`).

Why does that equation recover data? $H*B=0$ is true because encoding made the stripe valid. Splitting its columns does not change the equation:

$$\begin{aligned} F*BF + S*BS &= 0 \\ F*BF &= S*BS \quad (\text{subtraction equals addition in GF}(2^w)) \\ BF &= F^{-1}*(S*BS) \end{aligned}$$

F is a small matrix of coefficients for the unknown sectors; S is the corresponding matrix for sectors that survived. BF and BS are the sector **contents** — each entry represents an entire data region. T in this equation is just a temporary sector-sized result per parity equation; it is unrelated to the number of OpenMP threads called T in Milestone 2.

Why encoding uses the recovery path

The encoder designates some sectors as parity locations and treats them as the “missing” unknowns. Solving the same equation for those locations fills in parity that makes $H*B=0$. In `src/codec.h`, `ec_encode` aliases this recovery mechanism with the parity-sector locations as the faulty set. So the baseline code measures both encoding and decoding through the same mathematical machinery, with different unknown locations.

How much work does the baseline do?

The paper's work metric C counts calls over whole sector regions, rather than tiny arithmetic on coefficient matrices. $u(M)$ means the number of nonzero entries in M . For the normal sequence:

$$C = u(S) + u(F^{-1})$$

For the small paper example $SD^{\{1,1\}}_{\{4,4\}}(8|1,2)$ with failures $\{2,6,10,13,14\}$, $u(S)=22$, $u(F^{-1})=13$, so $C=35$. This is a count of region operations **for the small example**, not 35 milliseconds or a throughput result. Milestone 2's PPM path will take 29 calls on exactly the same example.

What we measured

For the performance benchmark, $n=r=16$, $z=1$ (the extra failures occupy one row), and all nine (m,s) pairs with m,s between 1 and 3. A codeword contains exactly 32 MiB, split into 256 sectors of 128 KiB each. The suite runs one warm-up and ten measured trials of **encoding and decoding** per configuration. Data is deterministically generated. The timed recovery call does not include wiping sectors, byte comparisons, syndrome checks, checksums, or writing CSV output.

The table reports **mean useful-data throughput**, calculated as data bytes divided by mean measured seconds, in MiB/s. It is from the Milestone 1 run recorded on an Intel Core Ultra 9 185H, GCC 11.4.0, -O2, with scalar arithmetic and no SIMD. These are the means documented in `README.md`; the per-trial source is `results/benchmark_results.csv`.

m	s	Field	Region calls C	Encode MiB/s	Decode MiB/s
1	1	GF(2 ⁸)	527	852.60	854.58
1	2	GF(2 ¹⁶)	783	634.68	631.51
1	3	GF(2 ³²)	1039	673.01	671.77
2	1	GF(2 ⁸)	828	444.98	447.81
2	2	GF(2 ¹⁶)	1084	399.48	401.21
2	3	GF(2 ³²)	1340	479.27	476.19
3	1	GF(2 ⁸)	1159	282.56	275.55
3	2	GF(2 ¹⁶)	1415	276.56	279.68
3	3	GF(2 ³²)	1671	352.91	349.51

What appears on slide 7: just the $s=1$ rows, with pairs of encode and decode bars and the rounded decoding values 855, 448, and 276 MiB/s. This holds the field width at eight bits while showing what changes with m . The nine-setting table is here for preparation, not projected in small type. Do not claim that throughput always decreases as s increases: changing s also changes the selected field width and its scalar arithmetic backend.

The separate correctness sweep covers 7,833 feasible (`configuration,z`) cases. That number describes the scope of the implementation checks; it is not a throughput and should not be treated as the main result of this talk.

How to explain this milestone in 30 seconds

“Our first task was to build the traditional SD decoder and keep it as a sequential reference. We used the published coefficient table to construct the parity equations, implemented GF arithmetic and matrix inversion, and recovered missing sectors using $T=S*BS$ followed by $BF=F^{-1}*T$. In the paper's small example the method makes 35 whole-sector calls. With a 32 MiB codeword, we measured sequential decode throughput across nine SD settings; for the $s=1$ settings shown on the slide it is about 855, 448, and 276 MiB/s as m goes from one to three. This unchanged path is what Milestone 2 is compared against.”

Questions a teammate may get

Why is $H*B=0$ true? The encoder chooses parity-sector values so that the full stripe satisfies exactly those equations. It is the definition of a valid encoded stripe, not an assumption about arbitrary input bytes.

Why is $u(M)$ a meaningful cost? Every nonzero coefficient in a matrix times a vector of sectors triggers a `mult_XORs` operation across a whole sector region. Zero coefficients skip that operation.

Why is the 32 MiB stripe not 32 MiB of useful data? Some sectors store parity. Throughput here divides *useful data bytes*, not the entire codeword, by the measured seconds.

Are M1's absolute MiB/s numbers directly comparable to the paper's? No. Our CPU and scalar GF backends differ from the paper's environment, which also uses optimized SIMD paths. These numbers establish the local baseline.

What is the difference between Milestone 1 and Milestone 2? M1 solves one coupled matrix sequentially in the normal order. M2 partitions SD failures, uses matrix-first for independent jobs, runs those jobs in OpenMP, and solves the dependent remainder after a barrier.

Source files and commands

- Core: `src/gf.c`, `src/matrix.c`, `src/sd_code.c`, `src/codec.c`.
- Configuration source: `data/FAST-Coefficients.txt`.
- Programs: `src/main.c`, `src/sweep.c`, `src/benchmark.c`.
- Measurements: `results/sweep_results.csv`, `results/benchmark_results.csv`.
- Reproduction: `make run` (paper example), `make sweep` (parameter grid), `make benchmark` (32 MiB throughput), `make test` (focused tests).